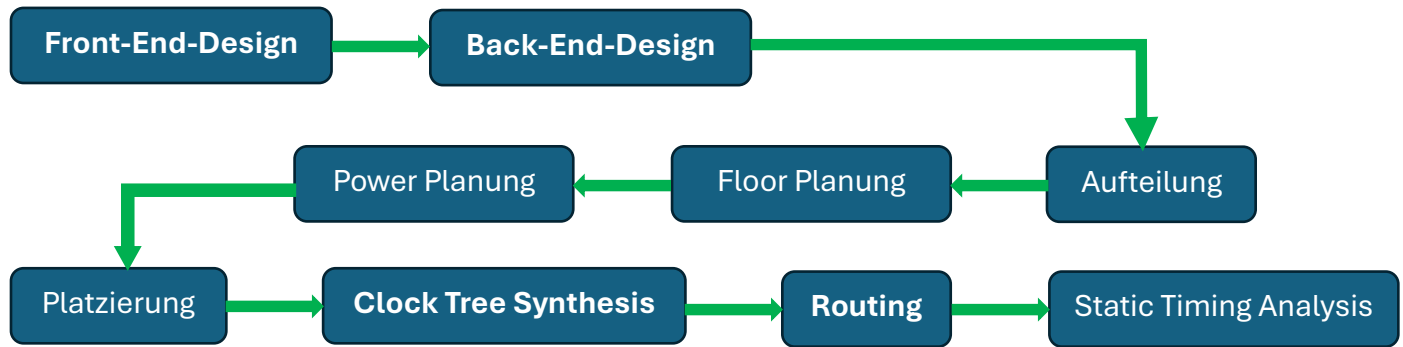




Aufteilung:

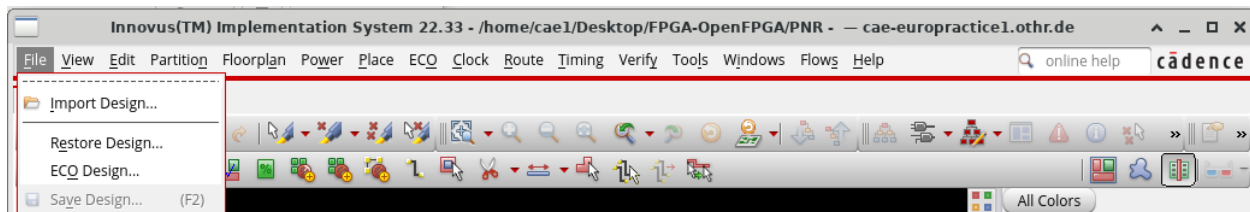
Es geht darum, ein komplexes/großes Design in kleine, überschaubare Blöcke oder Module zu zerlegen.

Entwurfsimport (nach Innovus):

Die meisten der nächsten Schritte werden von Innovus übernommen. Zunächst muss der Entwurf ordnungsgemäß in Innovus importiert werden. Außerdem können viele der folgenden Schritte, einschließlich des nachfolgenden Prozesses, von der Innovus-Konsole aus oder durch das Schreiben einer einfachen TCL-Datei durchgeführt werden.

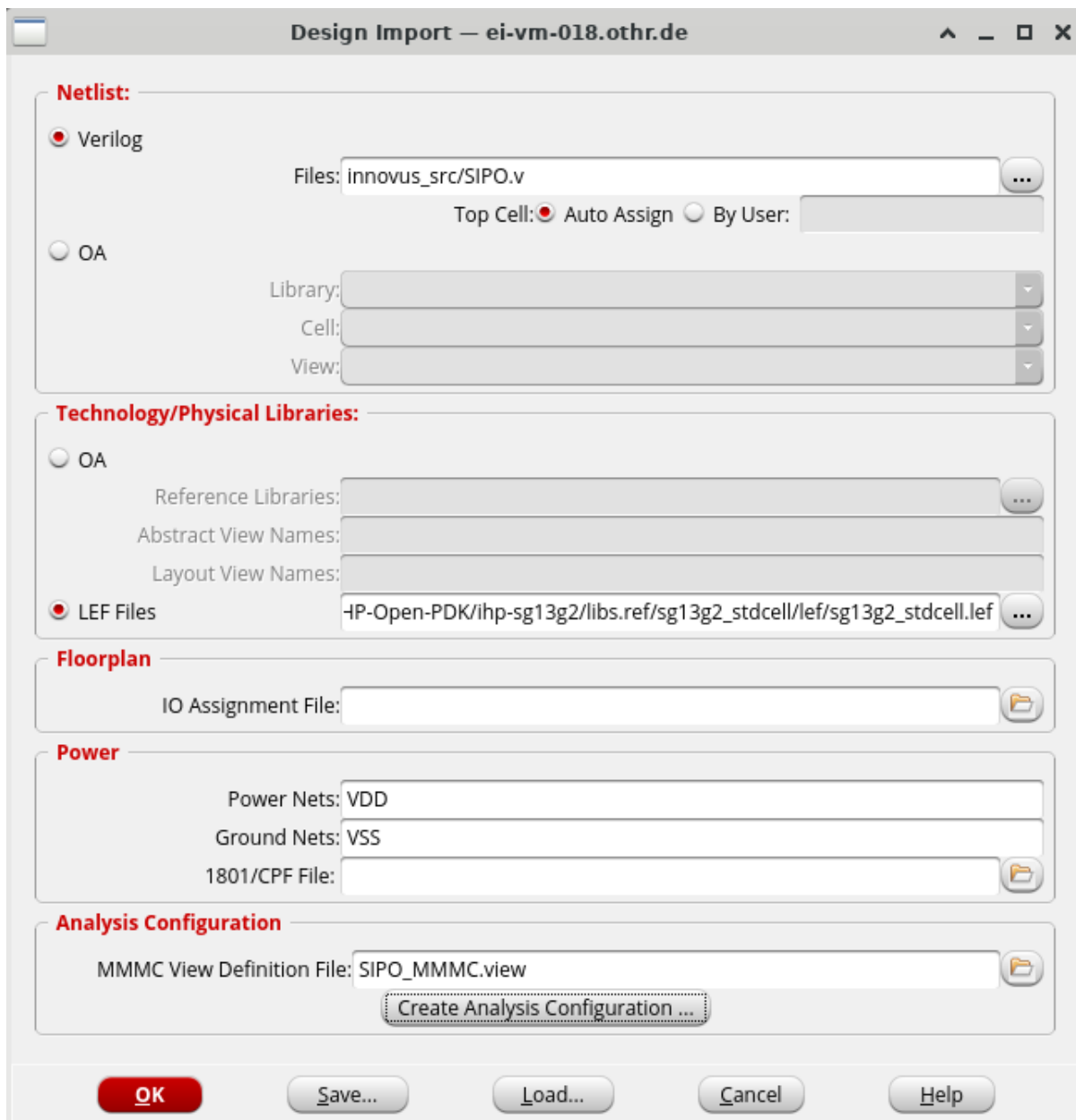
>> innovus

Import design:

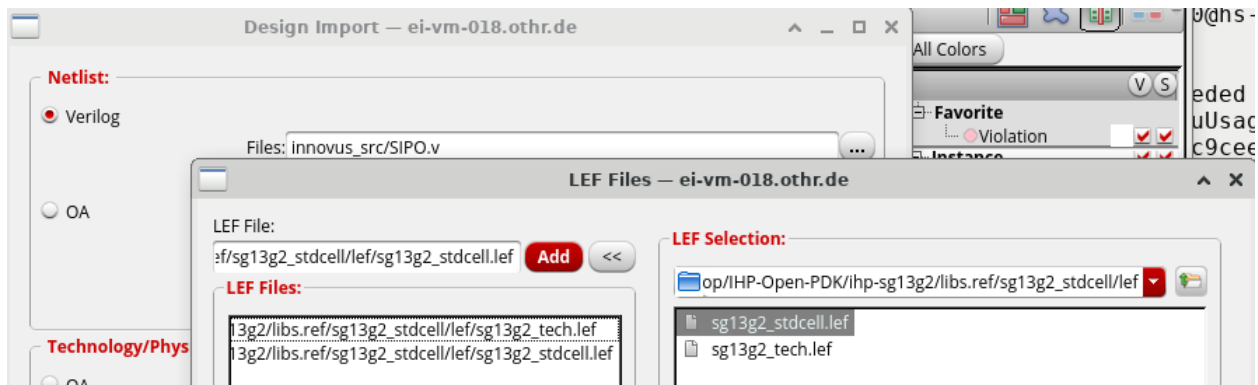


Während der Synthese exportiert der folgende Befehl die meisten der notwendigen Dateien für den Design-Import nach Innovus:

write_design -base_name innovus_src/SIPO



Beim Import einer LEF-Datei muss die Tech-Datei zuerst importiert werden.

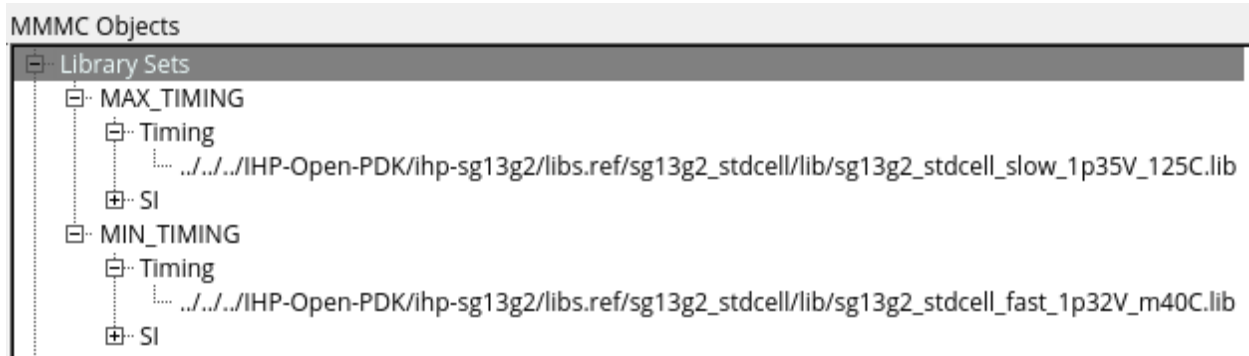
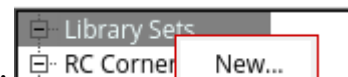


Die IO- Assignment-Datei ist optional (ist ein Muss, wenn IO-Pads verwendet werden). IO Pins/Pads können auch nach dem Import des Designs konfiguriert werden, entweder durch Laden der IO- Assignment-Datei oder durch Ausführen eines Skripts/Befehls oder einfach über das Layoutfenster.

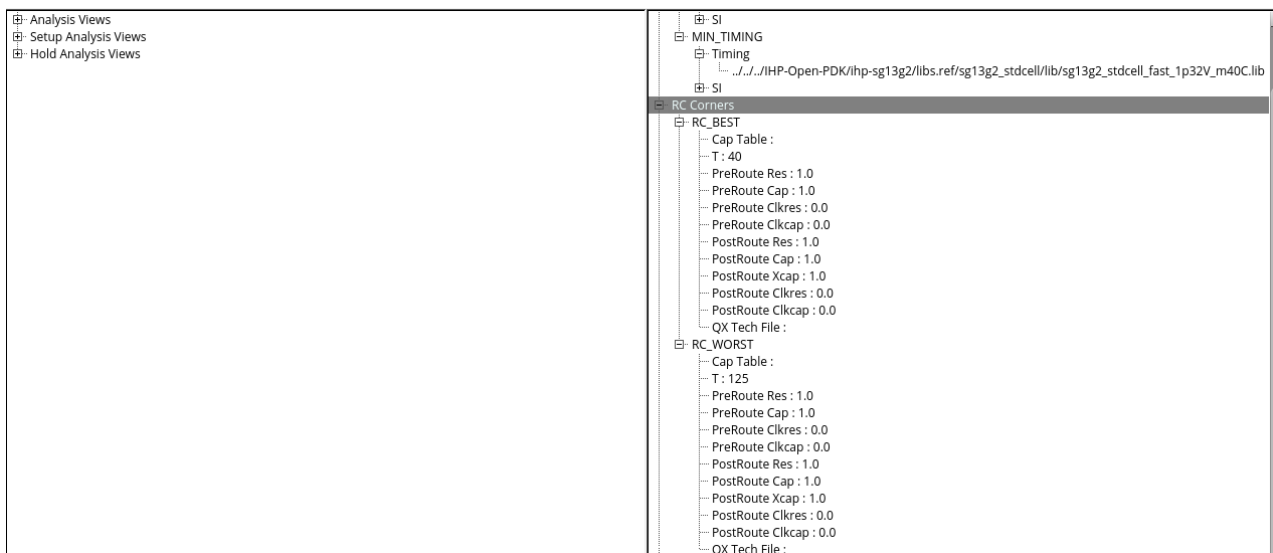
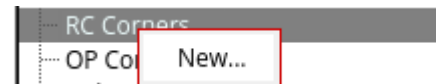
Die MMMC View Definition kann von Genus generiert werden, wenn sie während der Synthese geschrieben und verwendet wurde. Sie kann auch über das Popup-Fenster für den Design-Import konfiguriert oder einfach manuell geschrieben werden.

Auswählen 

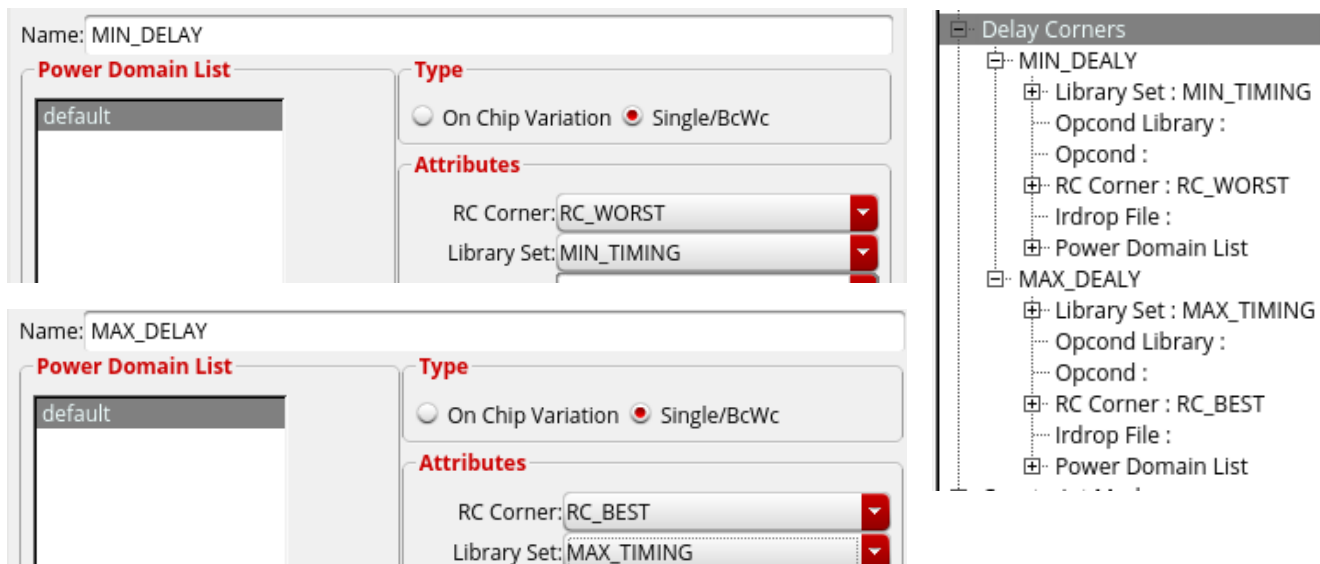
Erstellen der 2 „Library Sets“, eins für Max und eins für Min.



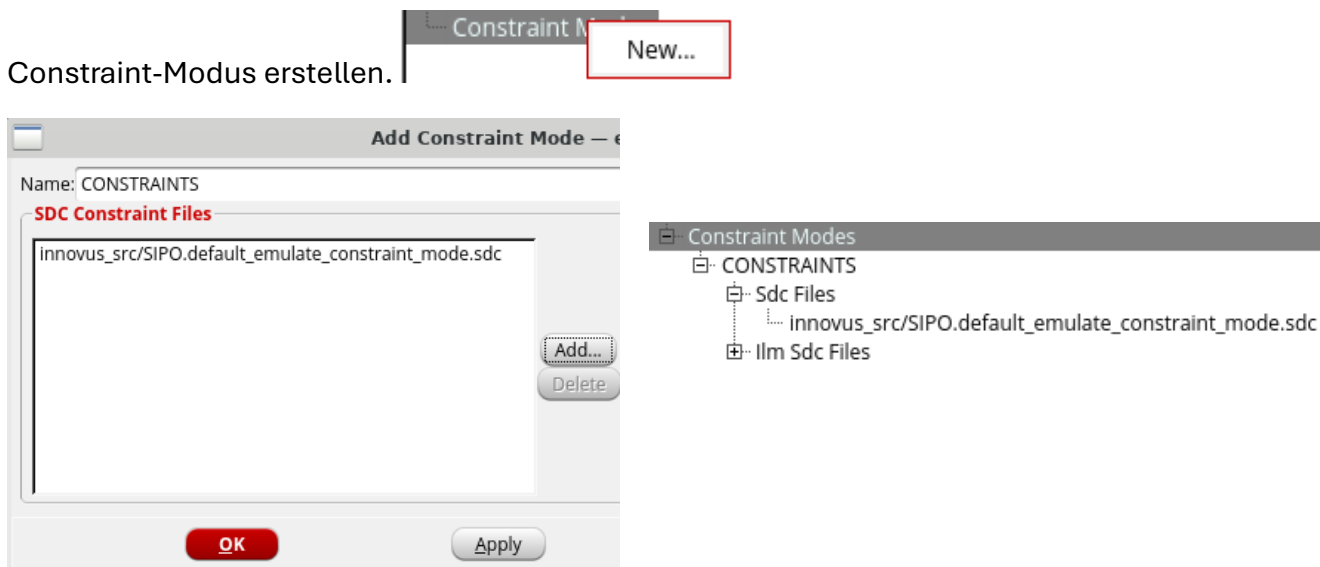
Erstellen 2 „RC Corner“, eine für die beste und eine für die schlechteste. In diesem Moment gibt es keine „Cap Table“ oder „QX Tech File“ für das PDK. Wenn vorhanden, kann importiert werden.



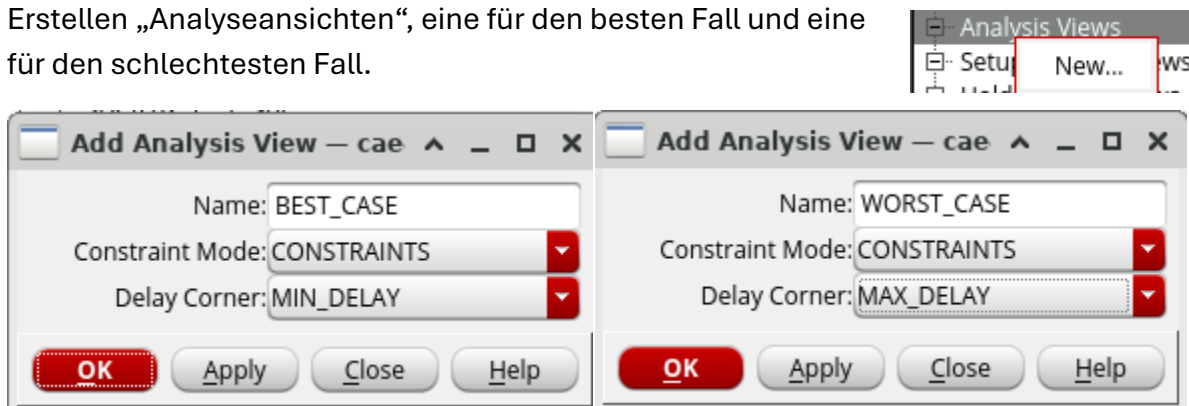
Erstellen Sie 2 „Verzögerungsecken“, eine für die maximale und eine für die minimale Verzögerung.



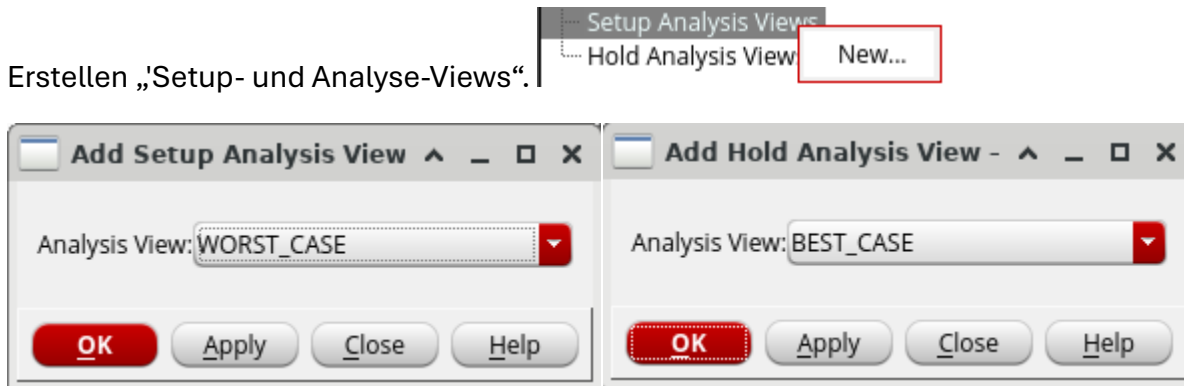
Constraint-Modus erstellen.



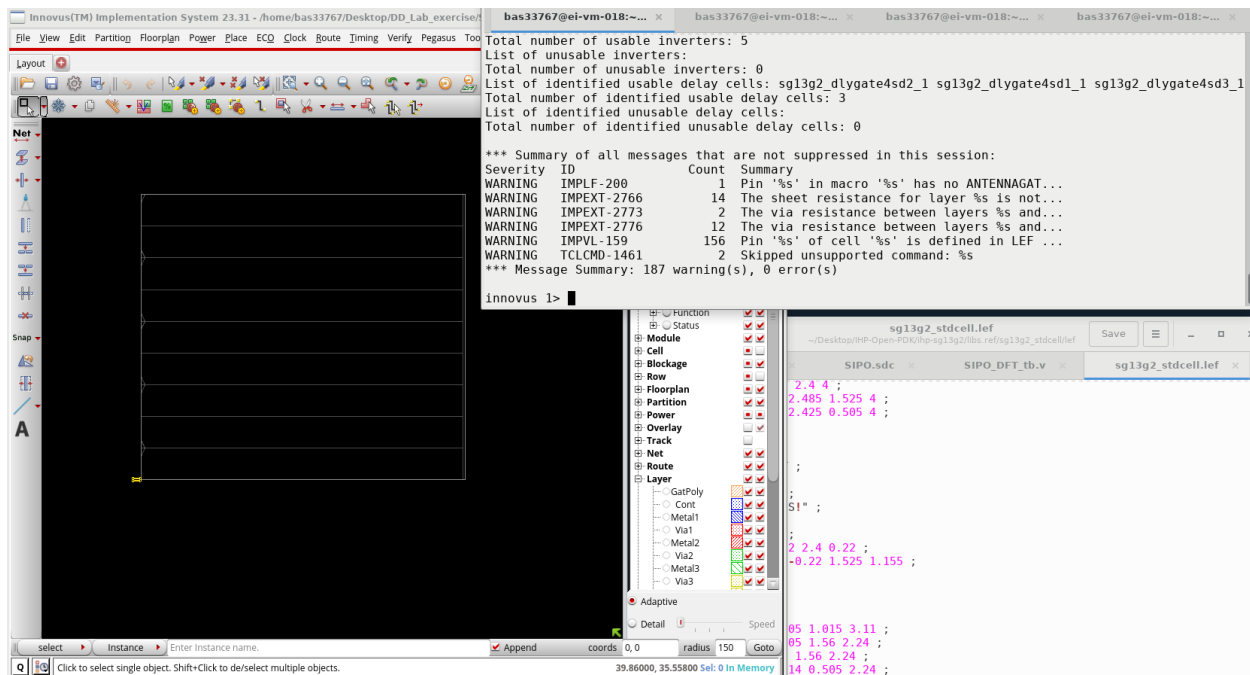
Erstellen „Analyseansichten“, eine für den besten Fall und eine für den schlechtesten Fall.



Erstellen „Setup- und Analyse-Views“.



Speichern und schließen. Nach dem Basisimport. Die „Design Import“-Einstellungen können auch gespeichert werden, so dass sie beim nächsten Mal einfach importiert werden können.

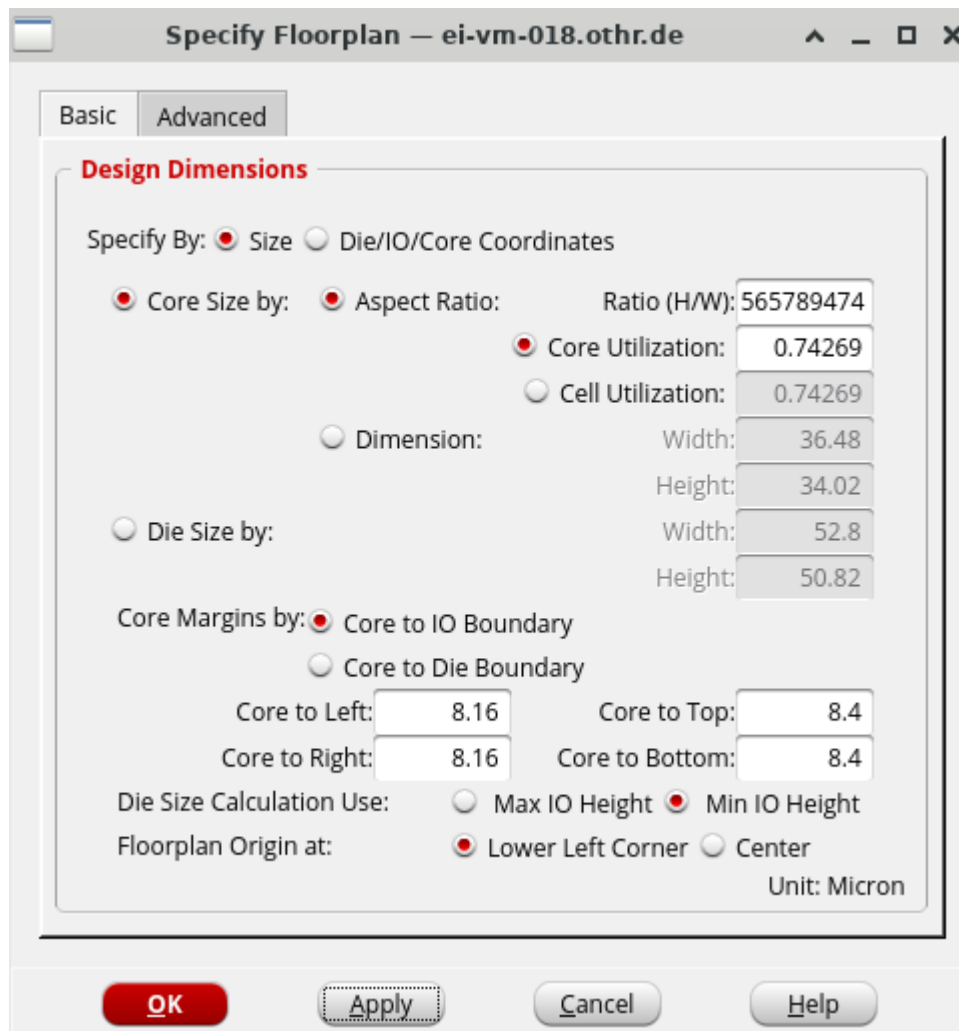
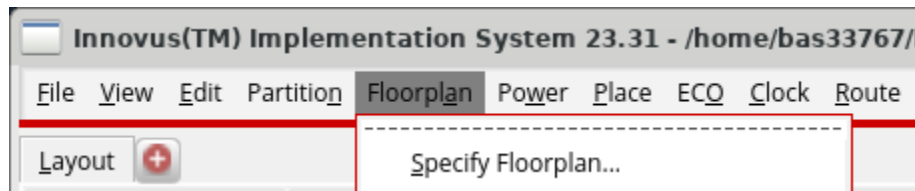


Hier werden einige kleinere Warnungen angezeigt, wenn es irgendwelche Fehler gibt, müssen diese im Protokoll angesprochen und behoben werden.

Die Hintergrundfarbe kann geändert werden:

>> setLayerPreference bg -color white

Floor Planung:



```
>> floorPlan -site CoreSite -r 1 0.75 8 8 8 8
```

Die Zahlen 8.16, 8.14 sind nicht zufällig, sondern basieren auf dem „Placement Grid Snap“.

```
>> get_snap_grid_info -type placement
```

```
innovus 1> get_snap_grid_info -type placement
{origin {0.0000000000,0.0000000000}} {pitch_x 0.4800000000} {pitch_y 0.4200000000}
innovus 2> □
```

e.g. $0.48 \times 17 = 8.16$, $0.42 \times 20 = 8.4$

Überprüfen des „Floorplan“:

checkDesign -floorplan

```
innovus 2> checkDesign -floorplan
Creating directory checkDesign.
Checking routing tracks.....
Checking other grids.....
Checking FINFET Grid is on Manufacture Grid.....

Checking core/die box is on Grid.....

Checking snap rule .....

Checking Row is on grid.....

Checking AreaIO row.....
Checking routing blockage.....
Checking components.....
Checking constraints (guide/region/fence).....
Checking groups.....
Checking Ptn Core Box.....

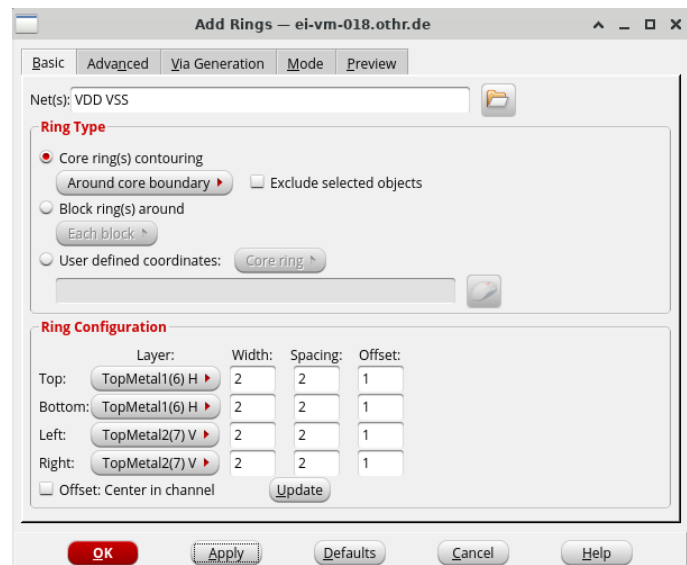
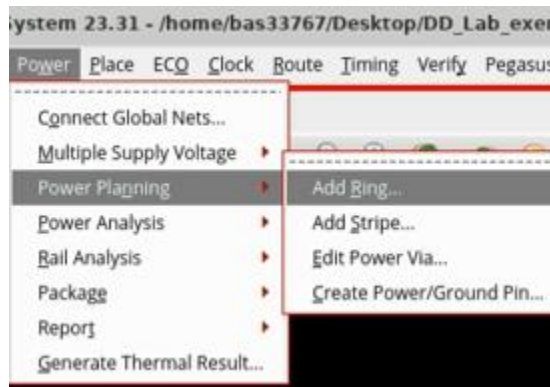
Checking Preroutes.....
No. of regular pre-routes not on tracks : 0
Design check done.
Report saved in file checkDesign/serial_to_parallel.main.htm.ascii
*** Message Summary: 0 warning(s), 0 error(s)
```

Setzen der Prozessknoten:

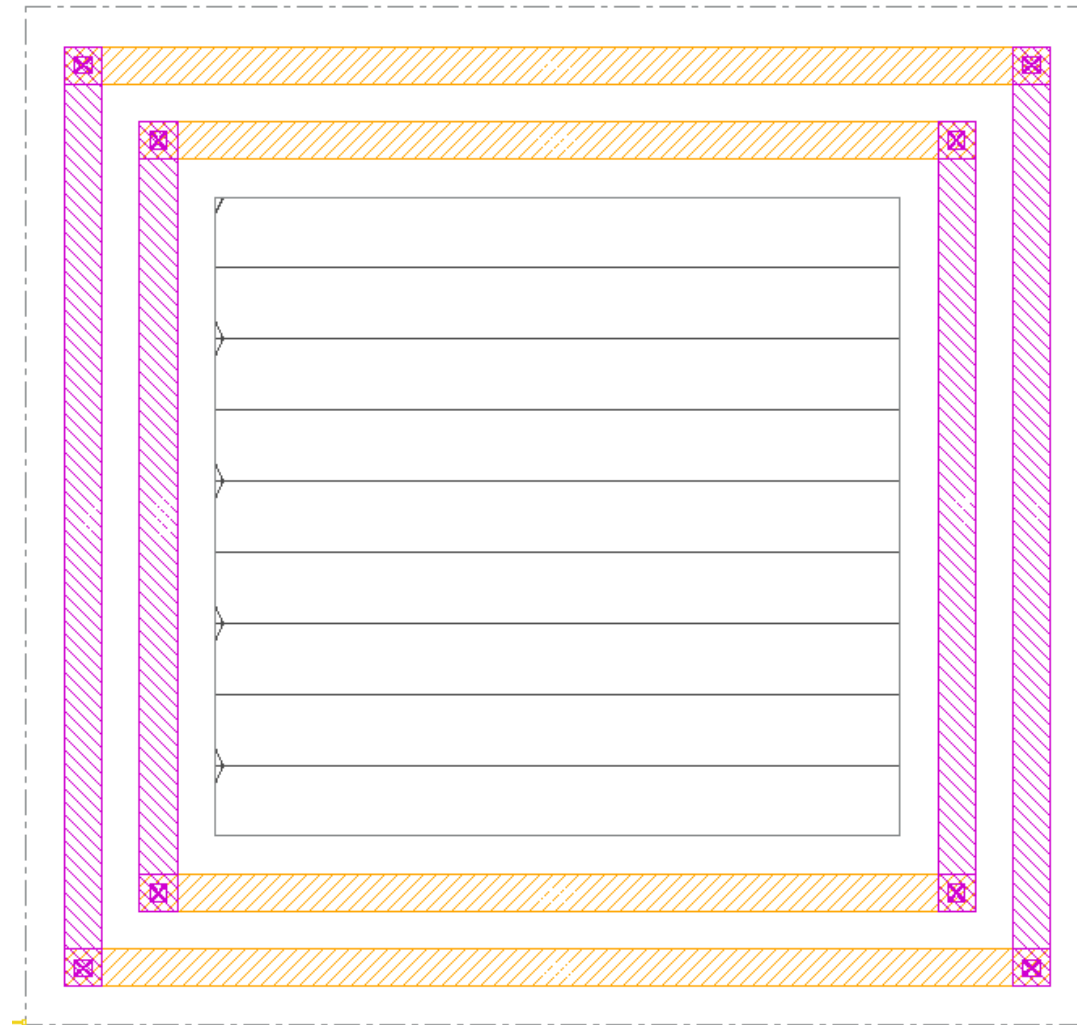
>> setDesignMode -process 130

Power Planung

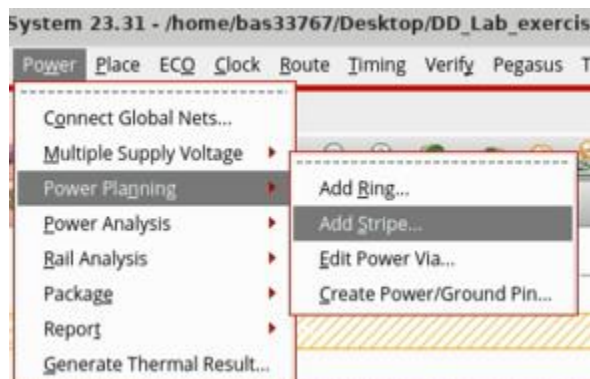
Add Ring:



```
>> addRing -nets {VDD VSS} -type core_rings -follow core -layer {top TopMetal1 bottom
TopMetal1 left TopMetal2 right TopMetal2} -width {top 2 bottom 2 left 2 right 2} -spacing {top
2 bottom 2 left 2 right 2} -offset {top 1 bottom 1 left 1 right 1} -center 0 -threshold 0 -
jog_distance 0 -snap_wire_center_to_grid None
```



Add Stripes:



Add Stripes — ei-vm-018.othr.de

Basic Advanced Via Generation Mode Preview

Set Configuration

Net(s): VDD VSS

Layer: TopMetal2(7) Directions: ☒ Vertical ☐ Horizontal

Width: 2 Spacing: 2

Set Pattern

☒ Set-to-set distance: 15 ☐ Number of sets: 1 ☐ Bumps

☐ Over P/G pins Pin layer: Top pin layer ▾ ☐ Pin Width:

☐ Master name: ☐ Selected blocks ☒ All blocks

☐ Over Physical Pins Pin layer: Top pin layer ▾ ☐ Pin Width:

Stripe Boundary

☒ Core ring ☐ Pad ring: Outer ▾ ☐ All domains

☐ Design boundary ☒ Create pins ☐ Each selected block/domain/fence

☐ Specify rectangular area

X1: Y1: X2: Y2:

☐ Specify rectilinear area

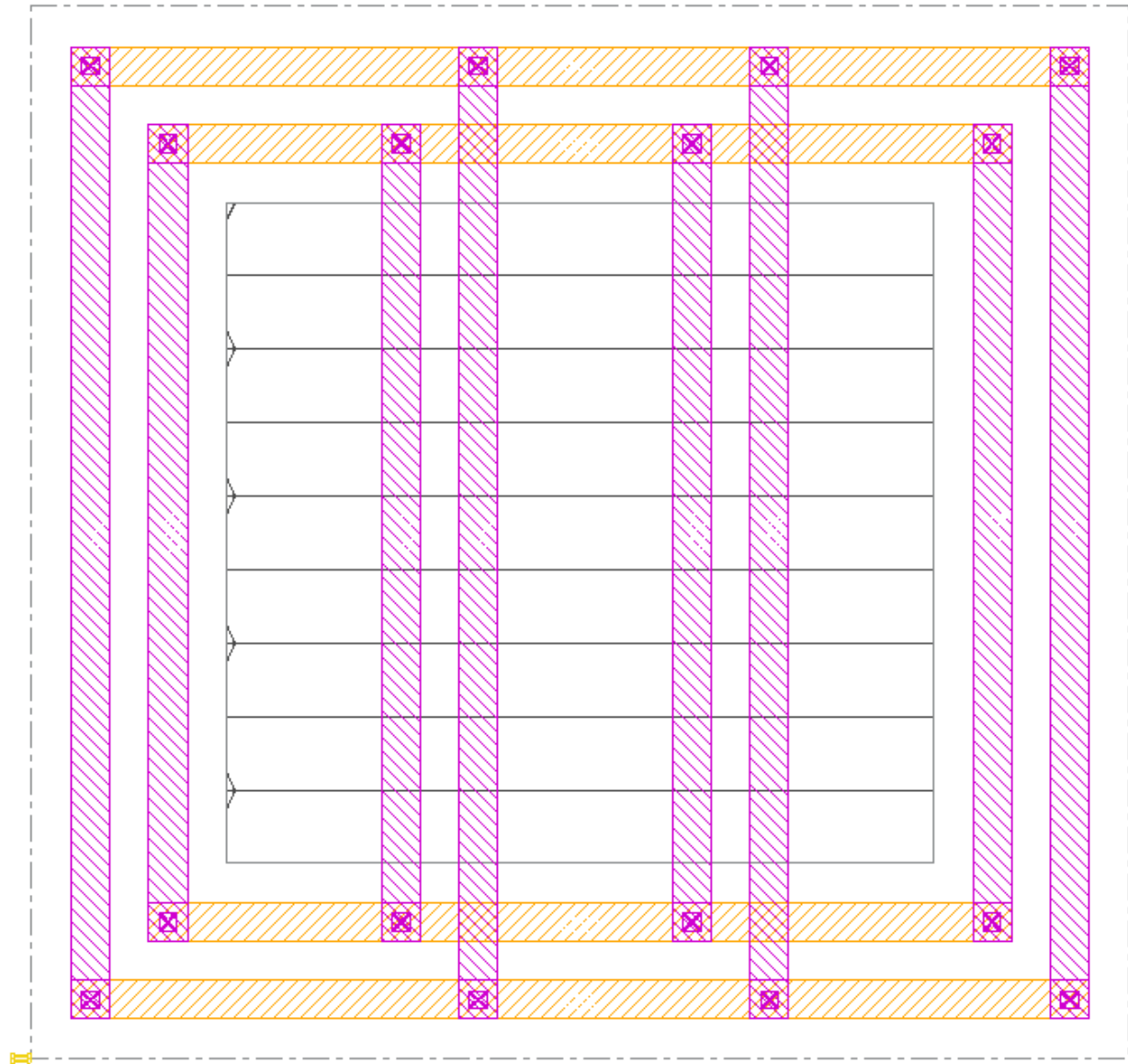
First/Last Stripe

Start from: ☒ Left ☐ Right ☐ Top ☐ Bottom

☒ Relative from core or selected area Start: 8 Stop:

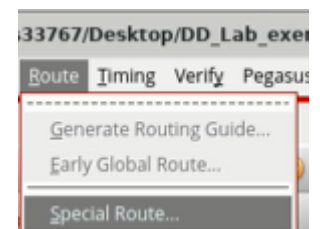
☐ Absolute Start: Stop:

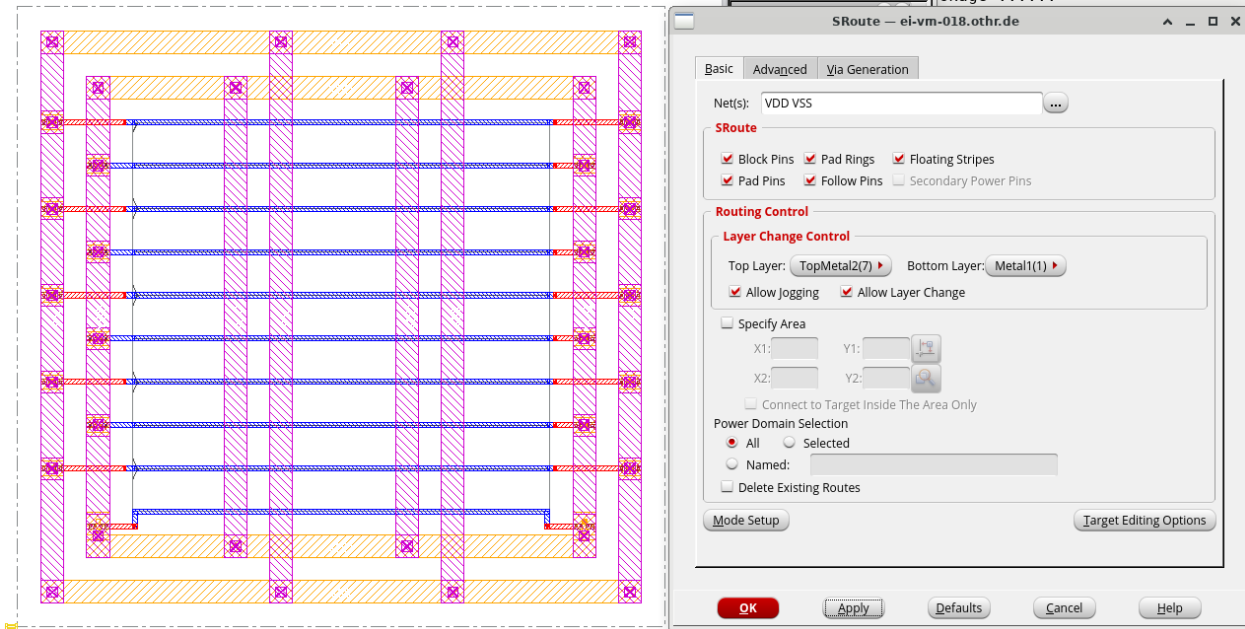
```
>> addStripe -nets {VDD VSS} -layer TopMetal2 -direction vertical -width 2 -spacing 2 -
set_to_set_distance 15 -start_from left -start_offset 8 -switch_layer_over_obs false -
max_same_layer_jog_length 2 -padcore_ring_top_layer_limit TopMetal2 -
padcore_ring_bottom_layer_limit Metal1 -block_ring_top_layer_limit TopMetal2 -
block_ring_bottom_layer_limit Metal1 -use_wire_group 0 -snap_wire_center_to_grid None
```



Special Route:

```
sroute -connect { blockPin padPin padRing corePin floatingStripe } -
layerChangeRange { Metal1(1) TopMetal2(7) } -blockPinTarget {
nearestTarget } -padPinPortConnect { allPort oneGeom } -padPinTarget {
nearestTarget } -corePinTarget { firstAfterRowEnd } -floatingStripeTarget {
blocking ring padring ring stripe ringpin blockpin followpin } -allowJogging 1
-crossoverViaLayerRange { Metal1(1) TopMetal2(7) } -nets { VDD VSS } -
allowLayerChange 1 -blockPin useLef -targetViaLayerRange { Metal1(1) TopMetal2(7) }
```





Logisches PG Connect erstellen:

```
>> globalNetConnect VDD -type pgpin -pin VDD -override -verbose -netlistOverride
```

```
>> globalNetConnect VSS -type pgpin -pin VSS -override -verbose -netlistOverride
```

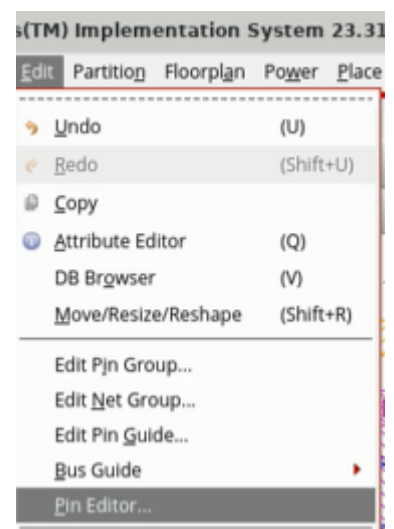
Platzierung

IO-Pin-Platzierung:

Die Platzierung der IO-Pads wird hier nicht behandelt. Eine ausführliche Anleitung findet sich jedoch im INNOVUS-Benutzerhandbuch. Viele Open-Source-PDKs sind nicht ohne Weiteres mit IO-Pads verfügbar, und wenn doch, dann ist die Timing-Library nicht vollständig. Dies führt zu vielen Fehlern und Warnungen während der Integration. Die Behebung solcher Fehler wird diese Übung komplexer und umfangreicher machen.

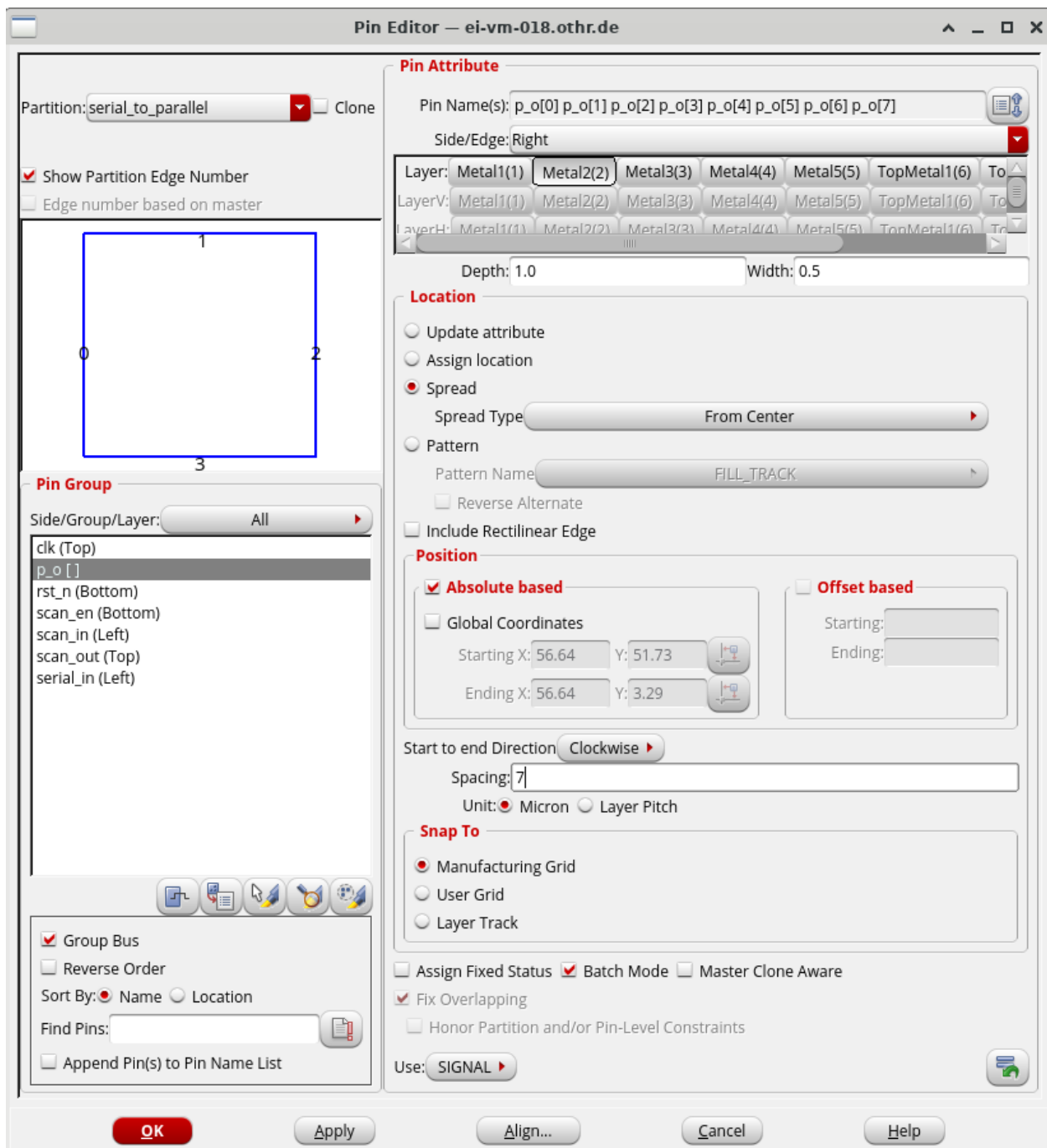
```
>> editPin -pinWidth 0.5 -pinDepth 1.0 -fixOverlap 1 -unit MICRON -
spreadDirection clockwise -side Left -layer 2 -spreadType center -
spacing 16.38 -pin {scan_in scan_out serial_in}
```

```
>> editPin -pinWidth 0.5 -pinDepth 1.0 -snap MGRID -fixOverlap 1 -unit MICRON -
spreadDirection clockwise -side Top -layer 2 -spreadType center -spacing 20.0 -pin { clk
{p_o[0]} {p_o[1]}}
```

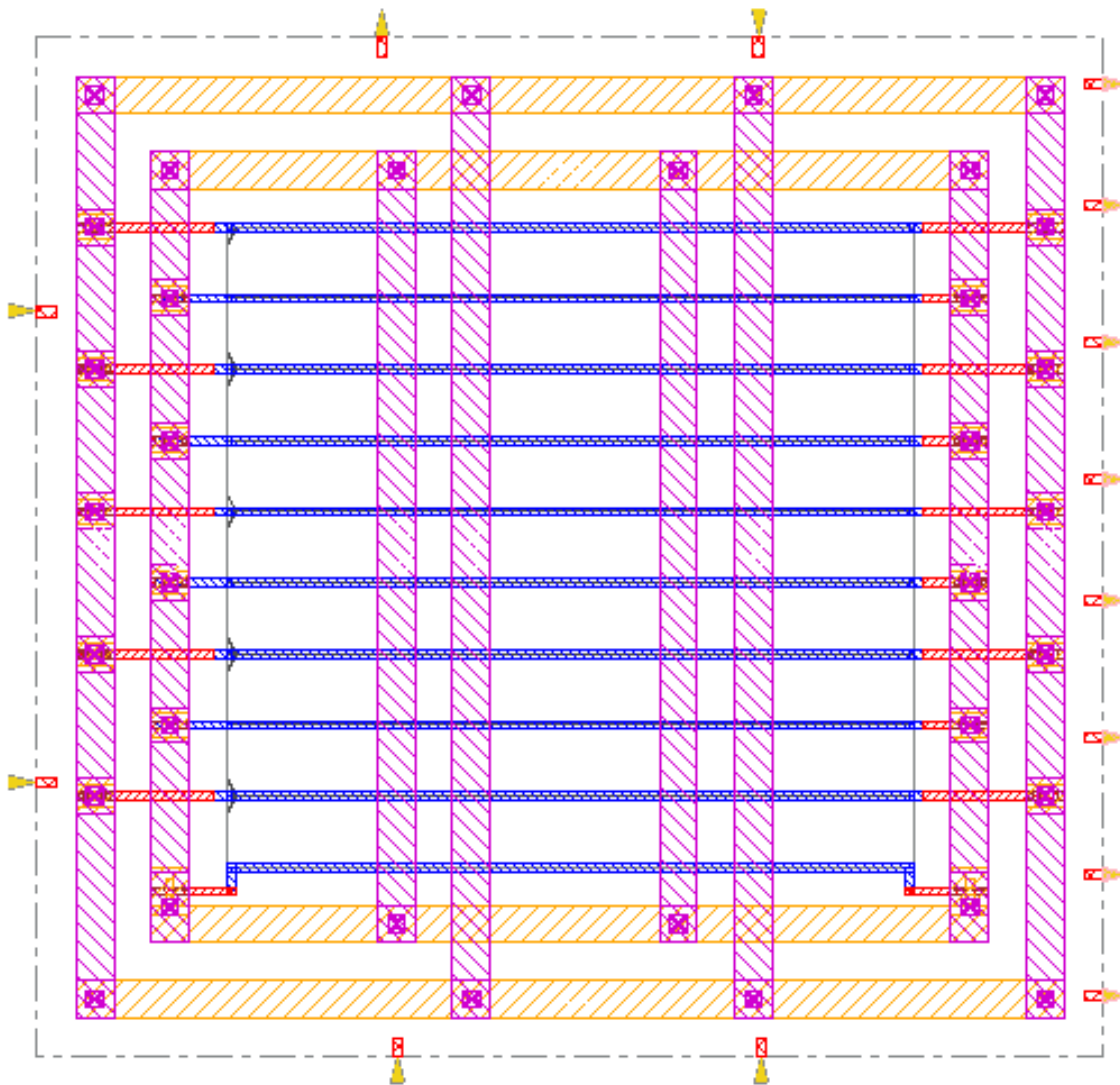


```
>> editPin -pinWidth 0.5 -pinDepth 1.0 -snap MGRID -fixOverlap 1 -unit MICRON -
spreadDirection clockwise -side Bottom -layer 2 -spreadType center -spacing 20 -pin
{{p_o[7]} rst_n scan_en}
```

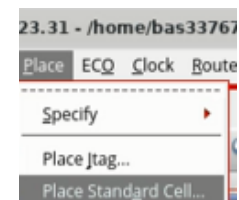
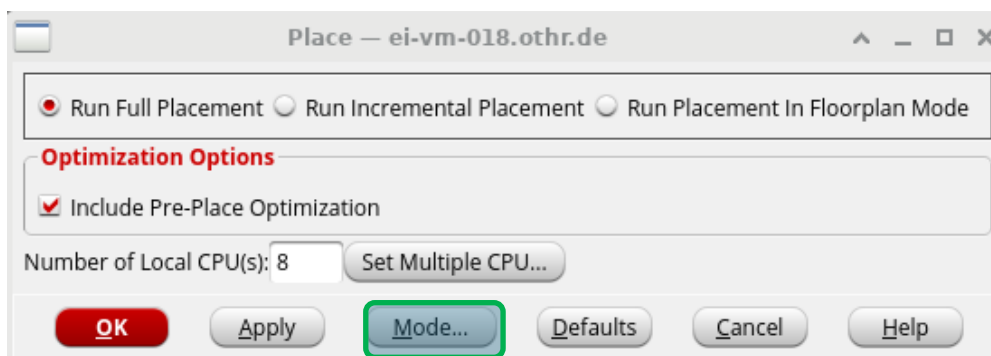
```
>> editPin -pinWidth 0.5 -pinDepth 1.0 -snap MGRID -fixOverlap 1 -unit MICRON -
spreadDirection clockwise -side Right -layer 2 -spreadType center -spacing 9.0 -pin
{{p_o[2]} {p_o[3]} {p_o[4]} {p_o[5]} {p_o[6]}}
```



IO-Pins können über die INNOVUS- Interface oder mit einem Skript für jede Seite (Oben, Rechts, Unten und Links) eingestellt werden. Vor der Anpassung sind alle Pins praktisch in der unteren linken Ecke des Die-Bereichs konzentriert.



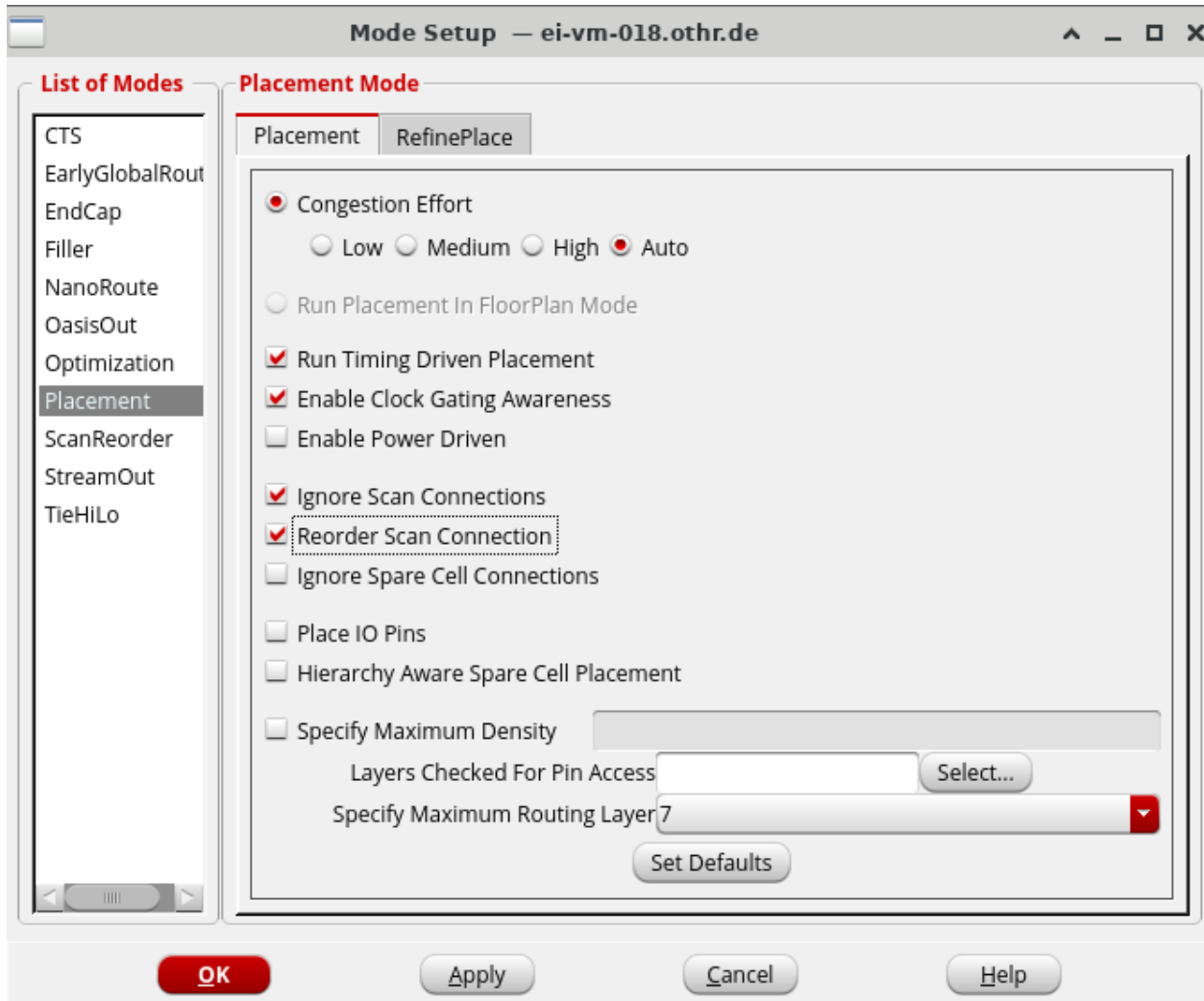
Place Standard Cells:



Da der Entwurf eine Scan-Chain enthält, muss die Scan-DEF-Datei aufgerufen werden.

```
>> defIn innovus_src/SIPO.scan.def
```

```
>> setScanReorderMode -compLogic true
```



Die Konfigurationen in der Schnittstelle können mit den unten stehenden Befehlen vorgenommen werden (nützlich bei Verwendung von TCL-Skripten):

```
setMultiCpuUsage -localCpu 8 -cpuPerRemoteHost 1 -remoteHost 0 -keepLicense true
```

```
setDistributeHost -local
```

```
setRouteMode -earlyGlobalHonorMsvRouteConstraint false -  
earlyGlobalRoutePartitionPinGuide true
```

```
setEndCapMode -reset
```

```
setEndCapMode -boundary_tap false
```

```
setNanoRouteMode -quiet -drouteAutoStop 0
```

```
setNanoRouteMode -quiet -drouteFixAntenna 0
```

```
setNanoRouteMode -quiet -droutePostRouteSwapVia {}
```

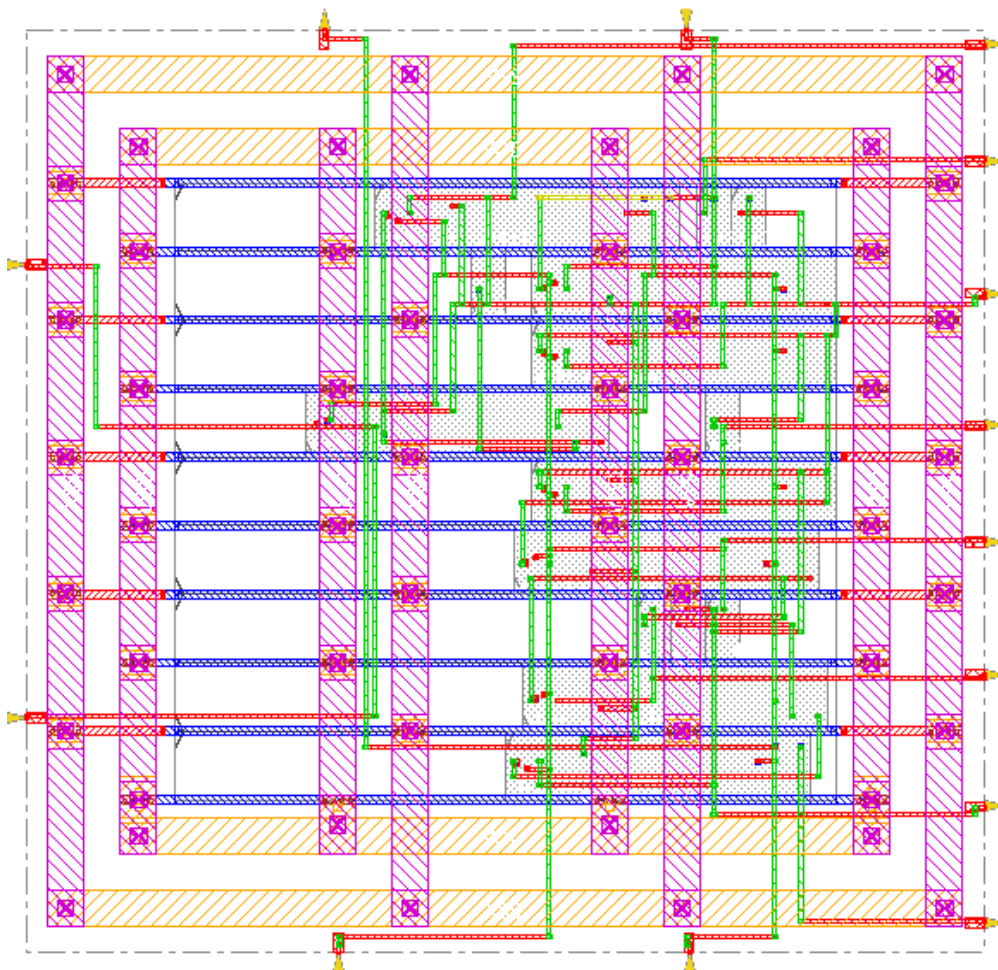


```

setNanoRouteMode -quiet -droutePostRouteSpreadWire 1
setNanoRouteMode -quiet -drouteUseMultiCutViaEffort {}
setNanoRouteMode -quiet -drouteOnGridOnly 0
setNanoRouteMode -quiet -routeIgnoreAntennaTopCellPin 0
setNanoRouteMode -quiet -timingEngine {}
setUsefulSkewMode -noBoundary false -maxAllowedDelay 1
setPlaceMode -reset
setPlaceMode -congEffort auto -timingDriven 1 -clkGateAware 1 -powerDriven 0 -
ignoreScan 1 -reorderScan 1 -ignoreSpare 0 -placeIOPins 0 -moduleAwareSpare 0 -
maxRouteLayer 7 -preserveRouting 1 -rmAffectedRouting 0 -checkRoute 0 -swapEEQ 0
setPlaceMode -fp false
place_design

```

Dies ist eine Proberoute und Platzierung. Sie wird in den nächsten Schritten aufgrund verschiedener Optimierungen angepasst.



Pre-CTS Optimization:

Timing vor CTS prüfen:

>> *timeDesign -preCTS*

Setup views included:
WORST_CASE

Setup mode	all	reg2reg	default
WNS (ns):	-1.383	-1.003	-1.383
TNS (ns):	-13.568	-7.451	-6.117
Violating Paths:	22	14	8
All Paths:	32	14	18

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	1 (1)	-0.202	1 (1)
max_tran	0 (0)	0.000	8 (8)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 45.029%
Routing Overflow: 0.00% H and 0.00% V

Der aktuelle Satz hat negative Slacks. Optimieren Sie den Entwurf:

>> *optDesign -preCTS*

Nach der Optimierung hat sich der gesamte negative Slacks verbessert, ist aber immer noch negativ.

Setup views included:
WORST_CASE

Setup mode	all	reg2reg	default
WNS (ns):	-0.204	-0.074	-0.204
TNS (ns):	-2.268	-0.794	-1.473
Violating Paths:	22	14	8
All Paths:	32	14	18

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	8 (8)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 78.655%
Routing Overflow: 0.00% H and 0.00% V

Manchmal kann die Überlastung einfach durch eine gleichmäßige Verteilung der IO-Pins/Pads verringert werden. Im Moment kann CTS durchgeführt werden, um zu prüfen, ob es sich verbessert.

Clock Tree Synthesis:

Während des Entwurfsimports wird das Protokoll eine nutzbare Liste von Buffer, Inverter und Delay Cells ansprechen.

```
580 List of usable buffers: sg13g2_buf_1 sg13g2_buf_16 sg13g2_buf_2 sg13g2_buf_4 sg13g2_buf_8
581 Total number of usable buffers: 5
582 List of unusable buffers:
583 Total number of unusable buffers: 0
584 List of usable inverters: sg13g2_inv_1 sg13g2_inv_16 sg13g2_inv_2 sg13g2_inv_4 sg13g2_inv_8
585 Total number of usable inverters: 5
586 List of unusable inverters:
587 Total number of unusable inverters: 0
588 List of identified usable delay cells: sg13g2_dlygate4sd2_1 sg13g2_dlygate4sd1_1
    sg13g2_dlygate4sd3_1
    ---_sg13g2_dlygate4sd3_1 ---_sg13g2_dlygate4sd3_1 ---_sg13g2_dlygate4sd3_1
```

```
>> set_ccopt_property buffer_cells {sg13g2_buf_1 sg13g2_buf_16 sg13g2_buf_2
sg13g2_buf_4 sg13g2_buf_8}
>> set_ccopt_property inverter_cells {sg13g2_inv_1 sg13g2_inv_16 sg13g2_inv_2
sg13g2_inv_4 sg13g2_inv_8}
>> set_ccopt_property delay_cells {sg13g2_dlygate4sd2_1 sg13g2_dlygate4sd1_1}
```

Nach CTS bleibt der negative „Slack“ bestehen. Dieser negative Schlupf resultiert daraus, dass der SDC für den Aufbau ohne DFT konzipiert wurde. Nach DFT wurden die FFs durch Scan-Chain-FFs ersetzt, und aufgrund der zusätzlichen Logik sind die Verzögerungen größer. Ohne die Scan-Chain war das Setup in der Lage, 400 pS (0,4 nS) Takt zu verarbeiten. Hierfür sollte der Takt 562 pS (0,562 nS) betragen. Nach der Korrektur der SDC-Datei:

```
>> create_ccopt_clock_tree_spec
>> clock_opt_design
```

Post CTS Optimization:

```
>> timeDesign -postCTS
>> optDesign -postCTS
```

Setup views included:
WORST_CASE

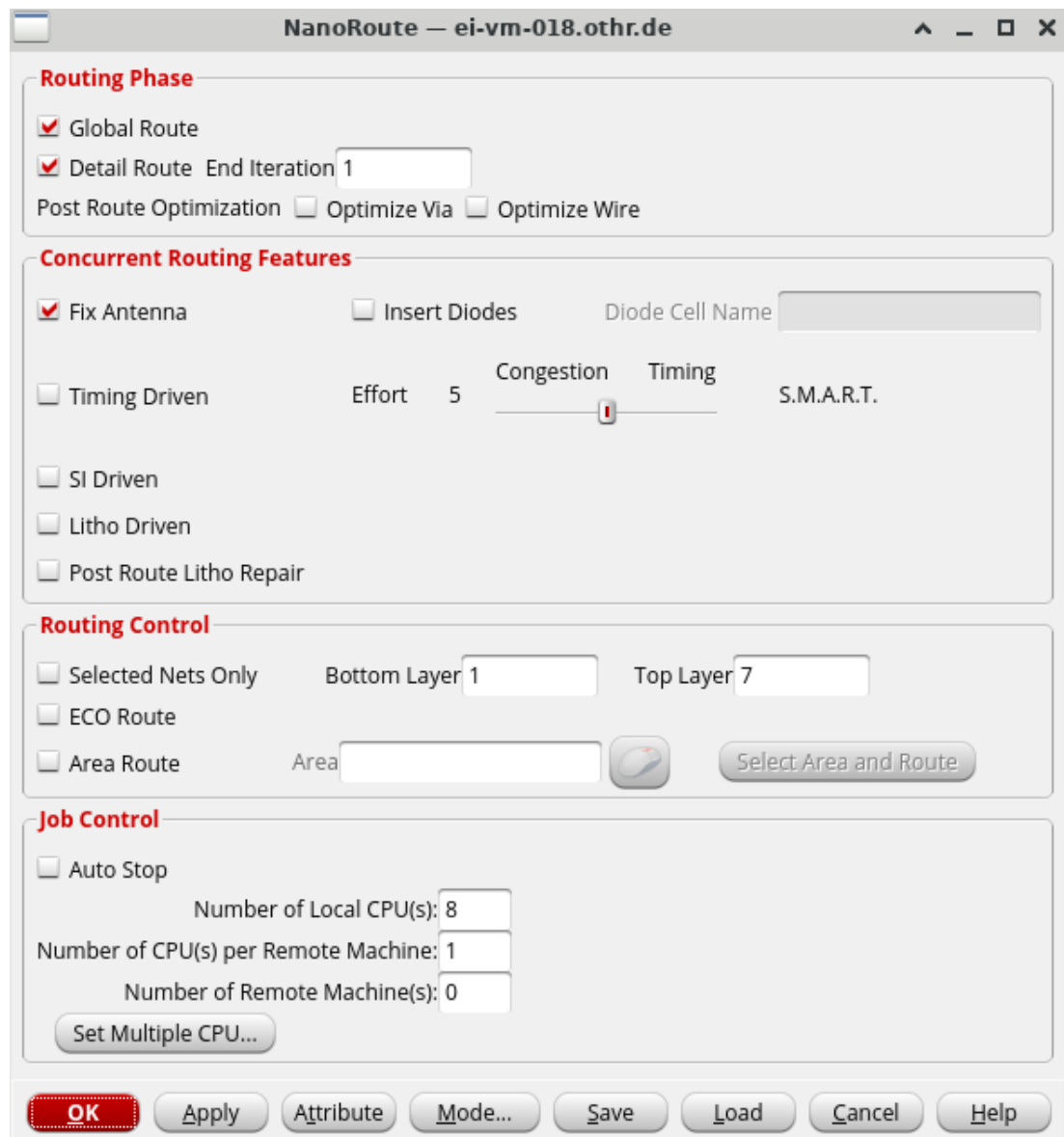
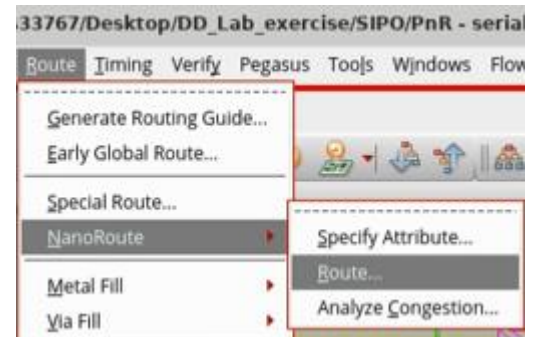
Setup mode	all	reg2reg	default
WNS (ns):	0.002	0.074	0.002
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	32	14	18

DRVs	Real		Total
	Nr nets (terms)	Worst Vio	Nr nets (terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	8 (8)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 74.854%

Routing:

```
setNanoRouteMode -quiet -drouteFixAntenna 1
setNanoRouteMode -quiet -routeInsertAntennaDiode 0
setNanoRouteMode -quiet -routeWithTimingDriven 0
setNanoRouteMode -quiet -routeWithEco 0
setNanoRouteMode -quiet -routeWithLithoDriven 0
setNanoRouteMode -quiet -droutePostRouteLithoRepair 0
setNanoRouteMode -quiet -routeWithSiDriven 0
setNanoRouteMode -quiet -drouteAutoStop 1
setNanoRouteMode -quiet -routeSelectedNetOnly 0
setNanoRouteMode -quiet -routeTopRoutingLayer 5
```



Wenn der Entwurf stark überlastet ist, kann es nach dem Routing zu Kurzschlüssen im Entwurf kommen. In diesem Fall können zusätzliche Schritte erforderlich sein. Wenn solche Fehler auftreten, werden die entsprechenden Netze mit dem folgenden Befehl gelöscht:

```
>> editDelete -regular_wire_with_drc
```

Mit dem folgenden Befehl werden die gelöschten Netze neu verlegt:

```
>> globalDetailRoute
```

Post-route optimization:

```
>> setDelayCalMode -SIAware false  
>> setAnalysisMode -analysisType onChipVariation  
>> timeDesign -postRoute  
>> time_design -post_route -hold  
>> optDesign -postRoute
```

Wenn alles fehlerfrei läuft, kann der Entwurf exportiert werden.

```
>> saveNetlist pnr_outputs/post_layout_SIPO.v  
>> all_hold_analysis_views  
>> all_setup_analysis_views  
>> write_sdf -ideal_clock_network pnr_outputs/post_layout_SIPO.sdf
```

Speichern GDS:

```
streamOut pnr_outputs/SIPO -mapFile <map_file_path> -libName <virtuoso_lib_name> -  
units 1000 -mode ALL
```

e.g.

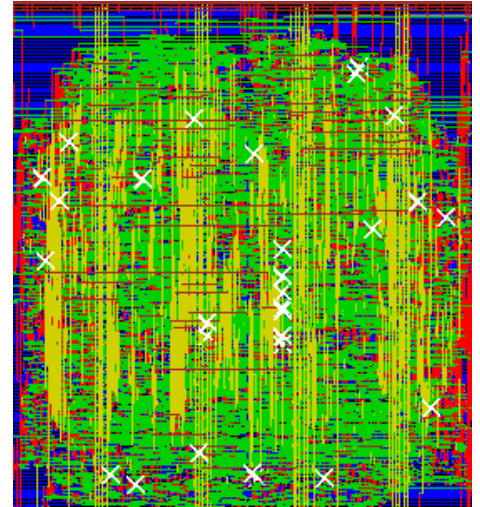
```
>> streamOut pnr_outputs/SIPO.gds -libName SIPO -units 1000 -mode ALL
```

Speichern SPEF:

```
rcOut -spef pnr_outputs/serial_to_parallel_RC_BEST.spef -rc_corner RC_BEST
```

```
rcOut -spef pnr_outputs/serial_to_parallel_RC_WORST.spef -rc_corner RC_WORST
```

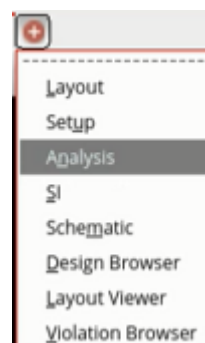
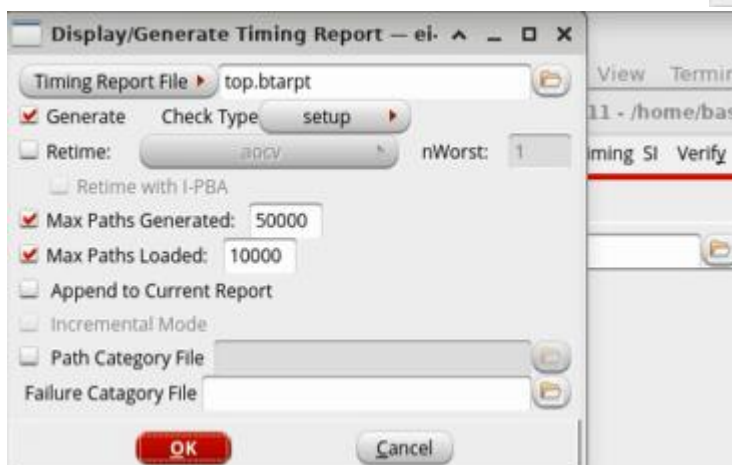
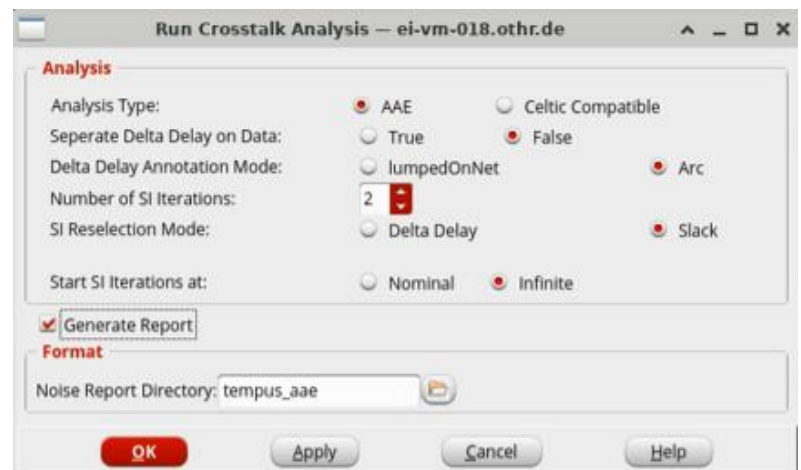
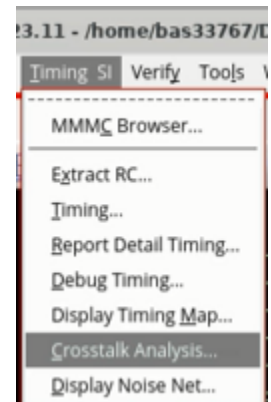
Physische Überprüfung:

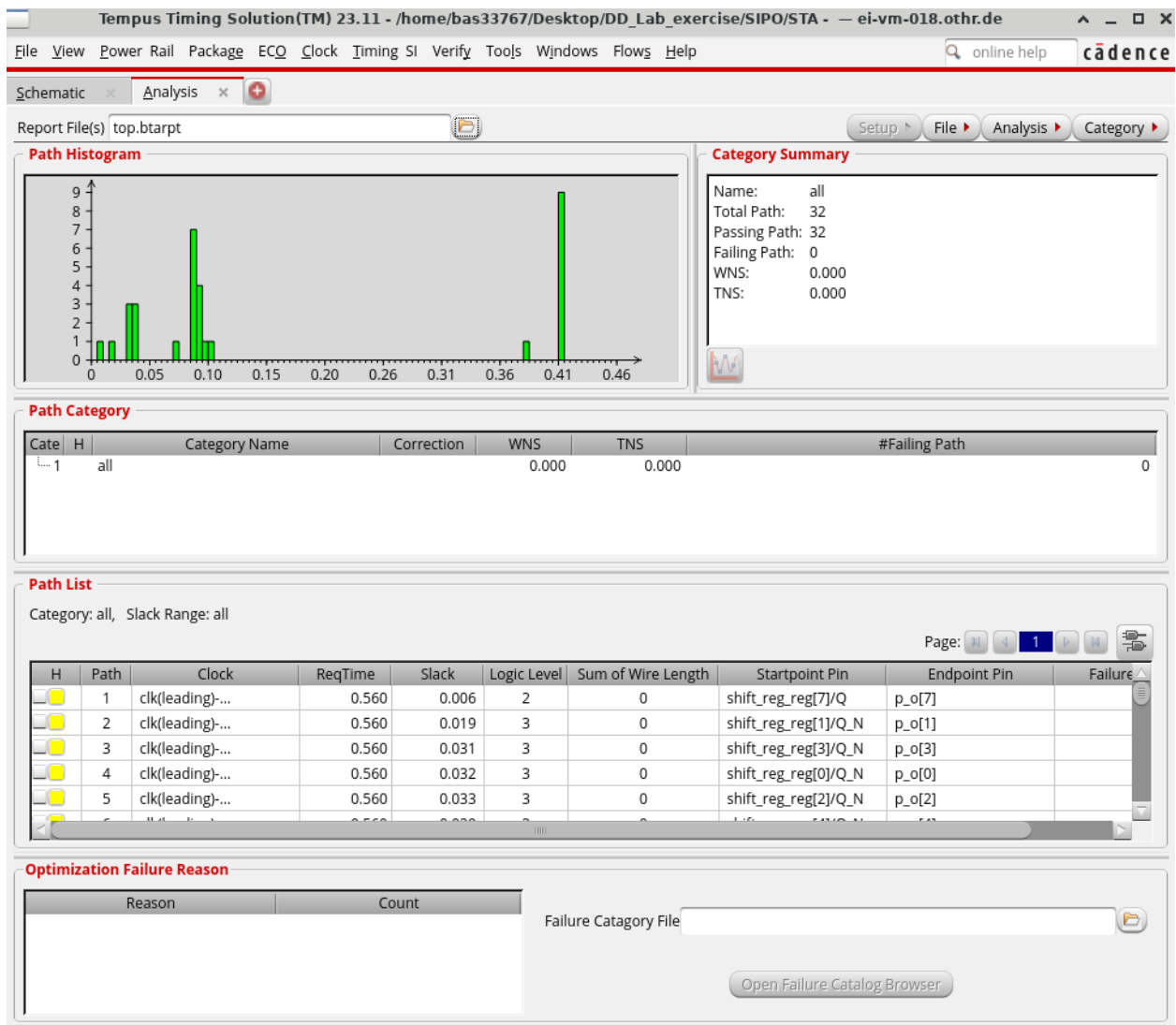


Nun steht die GDS-Datei zur Verfügung und kann in die entsprechende Virtuoso- Library importiert werden. DRC, LVS-Check und AMS-Analyse können wie bei einem typischen Analog-Design durchgeführt werden.

Static Timing Analysis

```
>>tempus
>> read_view_definition SIPO_MMMC.view
>> read_verilog pnr_outputs/post_layout_SIPO.v
>> set_top_module serial_to_parallel
>> read_spef -rc_corner RC_BEST
pnr_outputs/serial_to_parallel_RC_BEST.spef
>> read_spef -rc_corner RC_WORST
pnr_outputs/serial_to_parallel_RC_WORST.spef
>> set_si_mode -enable_delay_report true
>> set_si_mode -enable_glitch_report true
>> set_si_mode -enable_glitch_propagation true
>> update_timing -full
>> report_timing
```





Es liegen keine Verstöße vor. Wenn ja, können die ECO-Optionen geprüft werden.

Der gesamte Prozess wird mit IHP Open-PDK durchgeführt.

Als Übung können die gleichen Verfahren mit SkyWater 130, GPDK45 und GPDK90 durchgeführt werden.